Test Sessions and the Weekly Report

17 Oct 2026 · How Phoenix traces become a tool-accuracy table, how one day's session stays separate from the next, and how the week rolls up.
Fits steps 3, 4 and 7 of the Engagement Loop. Builder runs it. Client lead signs the numbers.

> **The report is not a screenshot of Phoenix.** Phoenix already records every model-based chat. The report is the approved test set joined to those traces, scored, then rolled up: run, then session, then week.

1. The four units

Unit	What it is	Name	Lives for
Test case	One question plus what "right" means	`T-014`	The whole engagement. Never renumber.
Run	One send of that question	`T-014-r2`	One reply
Session	One sitting. Same agent version and settings.	`20261017-1430`	A few hours
Week	Every session from Monday to Sunday	`2026-W42`	The report period

One Phoenix project per client and environment, for the whole engagement: `sharkk-petto-staging`. Sessions are labels inside that project. A new project every day makes the weekly join painful.

2. What every test case must say

The client approves this before any session (Engagement Loop, step 1).

Column	Example	Used to check
`test_id`	T-014	Join key
`category`	availability	Scorecard groups
`question`	Any vets open tomorrow at 5pm?	What we send
`expected_tool`	findAvailableDoctors	Right tool was called at least once
`allowed_tools`	findAvailableDoctors	Anything else is a wrong tool
`forbidden_tools`	searchServiceProviders	Instant fail
`must_include_args`	date=tomorrow	The fields that matter, not the whole JSON
`expected_outcome`	Lists only real vets with a free slot	The reply
`needs_tool`	yes	A no-tool answer is a fail. Use `no` for "hello".

3. Naming a session so days stay apart

Session id: `YYYYMMDD-hhmm` plus a short reason.

- `20261017-1430-baseline`
- `20261018-1015-after-prompt-fix`

Record the agent version with it: model, prompt version, `stepCountIs`. If any of those change, that is a new session, not a continuation.

How the label gets onto the trace

Today the code only stamps `functionId: vetmed-chatbot`, so two days in one project cannot be told apart. Pick one:

Method	How	Weekly combine
Works now	One Phoenix project per session: `sharkk-petto-staging-20261017-1430`. Set `PHOENIX_PROJECT_NAME` in the backend `.env` and restart.	Export each project, add a `session` column from the name, stack the CSVs.
Worth the small change	One project all week. Pass `session_id` and `test_id` as telemetry metadata on the chat call.	One export. Group by `session_id`.

> Also log every send: `session_id, test_id, run, sent_at, reply`. That is the join key. Matching on question text alone breaks when two cases share a question.

4. One session, start to finish

1. Phoenix is running. The backend prints `Phoenix tracing on` after the first chat.
2. Fresh chat per test. Clear history so case 12 does not inherit case 11.
3. Send each case 3 times (`r1`, `r2`, `r3`) for consistency.
4. Use a test patient and test clinics. Booking cases create real appointments.
5. Stop when every `test_id` has 3 recorded runs.

> Chats sent while Phoenix is off are not recorded and cannot be recovered. A plain “yes” or “no” that confirms a booking skips the model, so it is not traced. Score those from the reply itself.

5. From traces to the tool table

Export spans with `get_spans_dataframe` for that project. The dashboard caps at 1,000 spans, and a week can exceed that, so use an export script. Run the rows through `normalize_spans()`. Keep rows where `is_tool` is true. Join to the send log on time, or on `test_id` if it was stamped on the trace.

One row per test per run:

session	test_id	run	expected_tool	tools_called	args	result	answer	ms	status
20261017-1430	T-014	r1	findAvailableDoctors	findAvailableDoctors	pass	pass	pass	2100	pass
20261017-1430	T-014	r2	findAvailableDoctors	(none)	fail	n/a	fail	900	fail
20261017-1430	T-031	r1	bookAppointment	searchDoctorsClinics, bookAppointment	pass	pass	fail	6400	fail

Rules, in order

1. **Tool.** Pass if `expected_tool` is among `tools_called`. Fail if a forbidden tool appears. Fail if `needs_tool` is yes and the list is empty.
2. **Args.** Pass if every `must_include_args` field is present and matches. Do not fail on extra fields such as lat or lng.
3. **Tool result.** Pass if `tool_result` has `"ok": true`. For a booking, also `"booked": true` or `"needsConfirmation": true`, whichever the case expects. If this fails, the bug is the tool or the data, not the wording.
4. **Answer.** The reply names only things present in `tool_result`. A judge model does this. A person checks about 15% before the number is final (Engagement Loop, step 4).
5. **Loop.** The same tool 3 or more times in one trace is a fail, even if the expected tool was used.

6. Combining the week

Do not average sessions that used different agent versions. Show them as separate columns. A week total is only for the version you are willing to ship.

test_id	Mon 17th	Tue 18th	Week (same version)
T-014 tool	2/3	3/3	5/6 = 83%
T-014 answer	2/3	3/3	5/6
All cases, right tool	81%	89%	85%
All cases, grounded	90%	94%	92%

The weekly report (see *Agent Report Template*) is this table plus the finding cards. A finding is a pattern, not a single miss: "T-014 fails by skipping the tool in 4 of 18 runs this week." Evidence is the trace id. Frequency is the week fraction.

> **Re-test (Engagement Loop, step 7)** is a new session with the same `test_id` s. Name it `20261020-1600-retest` . The before/after page is last week's session versus this one. Never edit an old session's rows.

7. Done when

- Every approved test case has 3 runs in a named session.
- The results CSV has one row per run, and the scorecard totals equal that CSV.
- Tool, arguments, tool result, and answer are separate columns, so a wrong tool is not blamed on the wording.
- The week table shows each session, and mixes versions only when that is labelled.
- The report quotes session ids and trace ids, not screenshots of Phoenix.